

Глава X. QinQ в Debian 12: Полная реализация, ограничения и поведение в GNS3

Введение

QinQ (802.1ad) — это технология, позволяющая инкапсулировать один VLAN-тег (C-tag) внутри другого (S-tag). Она используется провайдерами для изоляции клиентских VLAN, транзита большого количества служб или построения L2VPN.

В аппаратных маршрутизаторах (Cisco, Eltex, Mikrotik) QinQ реализован на уровне ASIC/FPGA, что влияет на то, как кадры обрабатываются и что можно увидеть в пакетном захвате.

Linux, в отличие от них, — система общего назначения. Поэтому его поведение при QinQ **отличается**, особенно заметно это в GNS3.

Эта глава описывает реализацию QinQ в Debian 12, особенности работы, различия с Eltex vESR и типичную схему лаборатории.

1. Архитектура VLAN в Linux

Linux использует **полностью программную** реализацию VLAN-тегов через netlink-протокол и драйвер 8021q.

Важно понимать:

В Linux tcpdump видит весь реальный кадр — со всеми тегами.

Никакой аппаратный offload не скрывает C-tag или S-tag.

Поэтому при захвате трафика на физическом интерфейсе, например:

```
tcpdump -vvv -i ens5 -e -n
```

вы увидите оба тега:

- внешний S-tag (например, VLAN 100),
- внутренний C-tag (например, VLAN 10).

На аппаратных РЕ-устройствах (Eltex, Cisco):

- ASIC вырезает теги до того, как трафик попадёт в CPU,
- поэтому capture показывает **только тот тег**, что нужен контрол-плэне.

2. Типовая схема QinQ в Debian/GNS3

Маршрутизаторы R1 и R2 соединены trunk-каналом через физические интерфейсы:

R1 ens5 <—> ens5 R2

На каждом маршрутизаторе:

- внешний VLAN: **S-tag 100**
- внутренние VLAN: **10 и 20**
- на R2 оба внутренних VLAN бриджируются, чтобы получить L2-доступ к DHCP на R1

3. Постановка задачи

-настроить L2 сеть между двумя виртуальными машинами с ОС Debian 12.4 в симуляторе GNS3. Виртуальные машины Debian 12.4 получить с сайта GNS3. Провести их первоначальную настройку и обносить пакеты. Сеть построить в соответствии с схемой на рисунке 19-6.

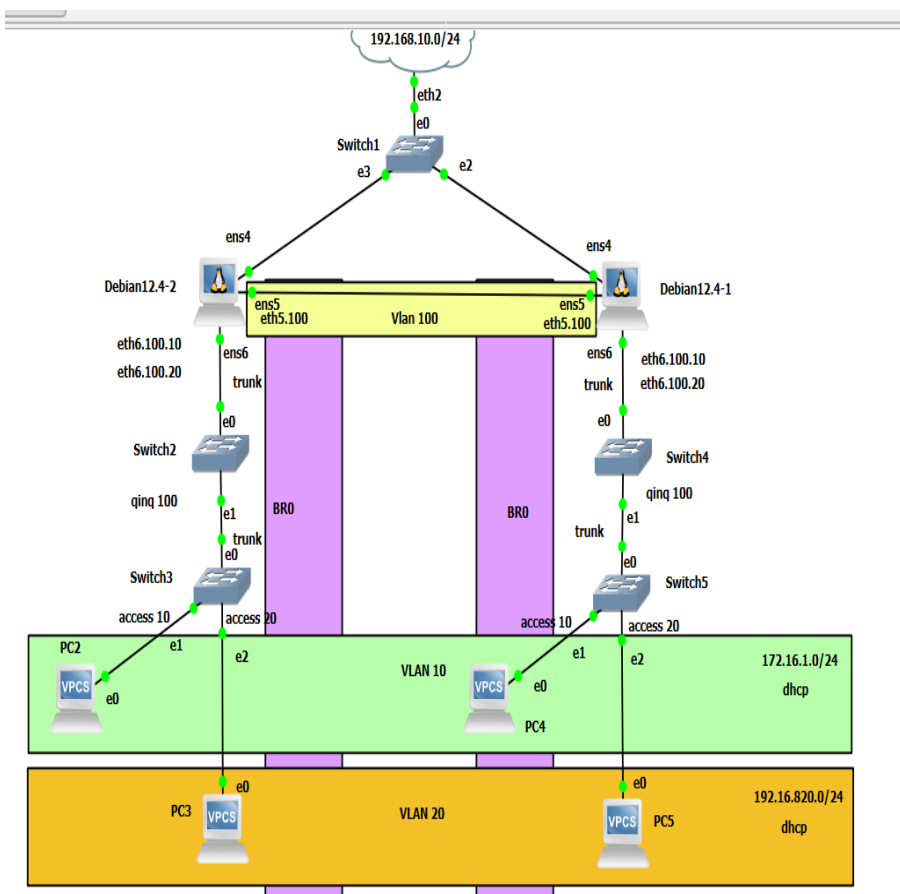


Рисунок 19-6. Схема сети Qinq.

Предварительные настройки сети в VMware Workstation Pro

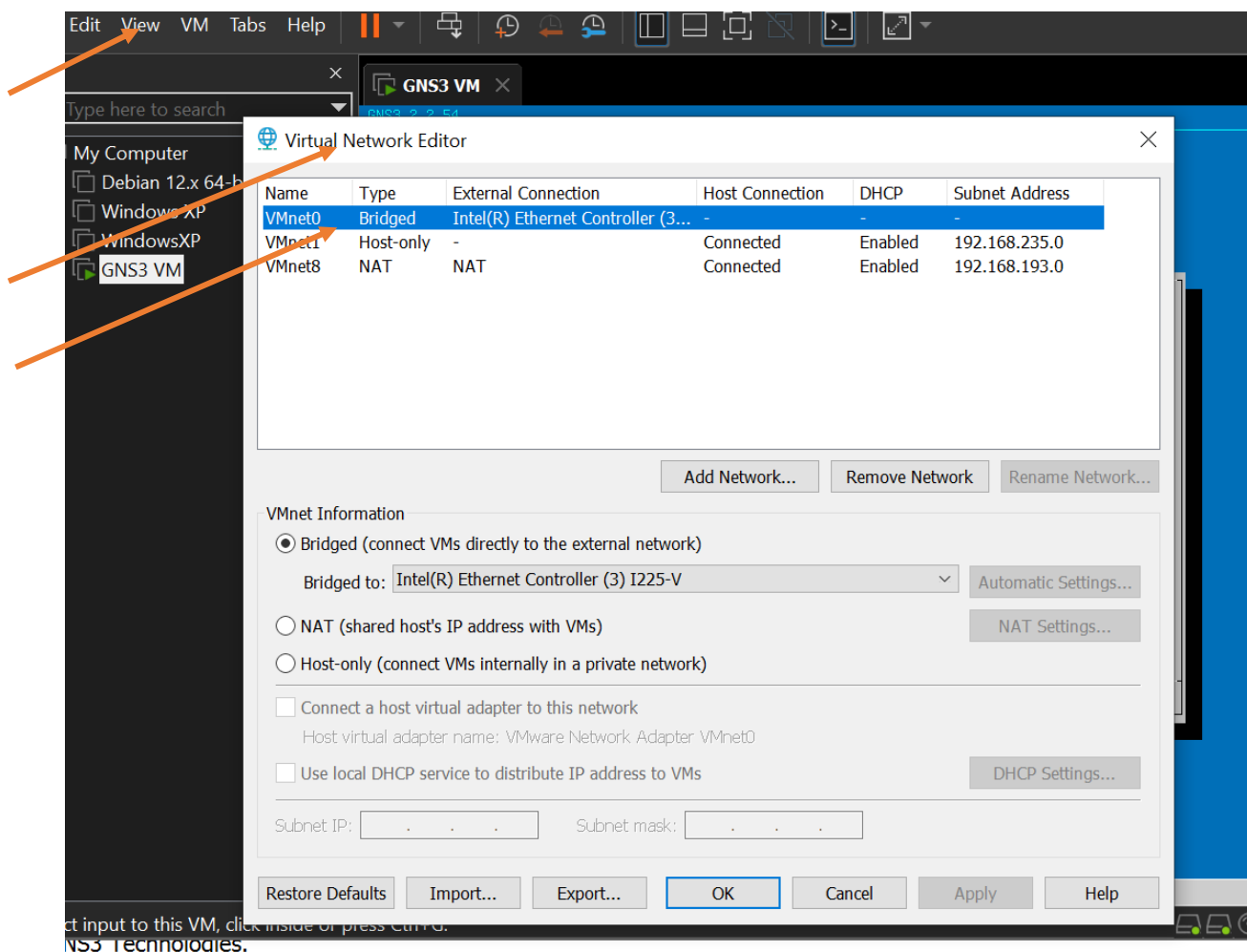


Рисунок 19-7. Настройка сетевых интерфейсов в программе VMware Workstation Pro.

Для возможности обновления пакетов и утилит в виртуальной машине GNS3 и виртуальных устройств на основе Linux нужно включить сетевой мост к сетевой карте основного ПК. Стрелками красного цвета показаны необходимые пункты меню для включения. В приведенном рисунке 19-7 показан снимок экрана программы, запущенной с административными привилегиями. Далее необходимо вызвать на настройки виртуальной машины и настроить адаптеры сети, как показано на рисунках 19-8 и 19-9.

Красными стрелками выделены необходимые пункты меню для этой работы.

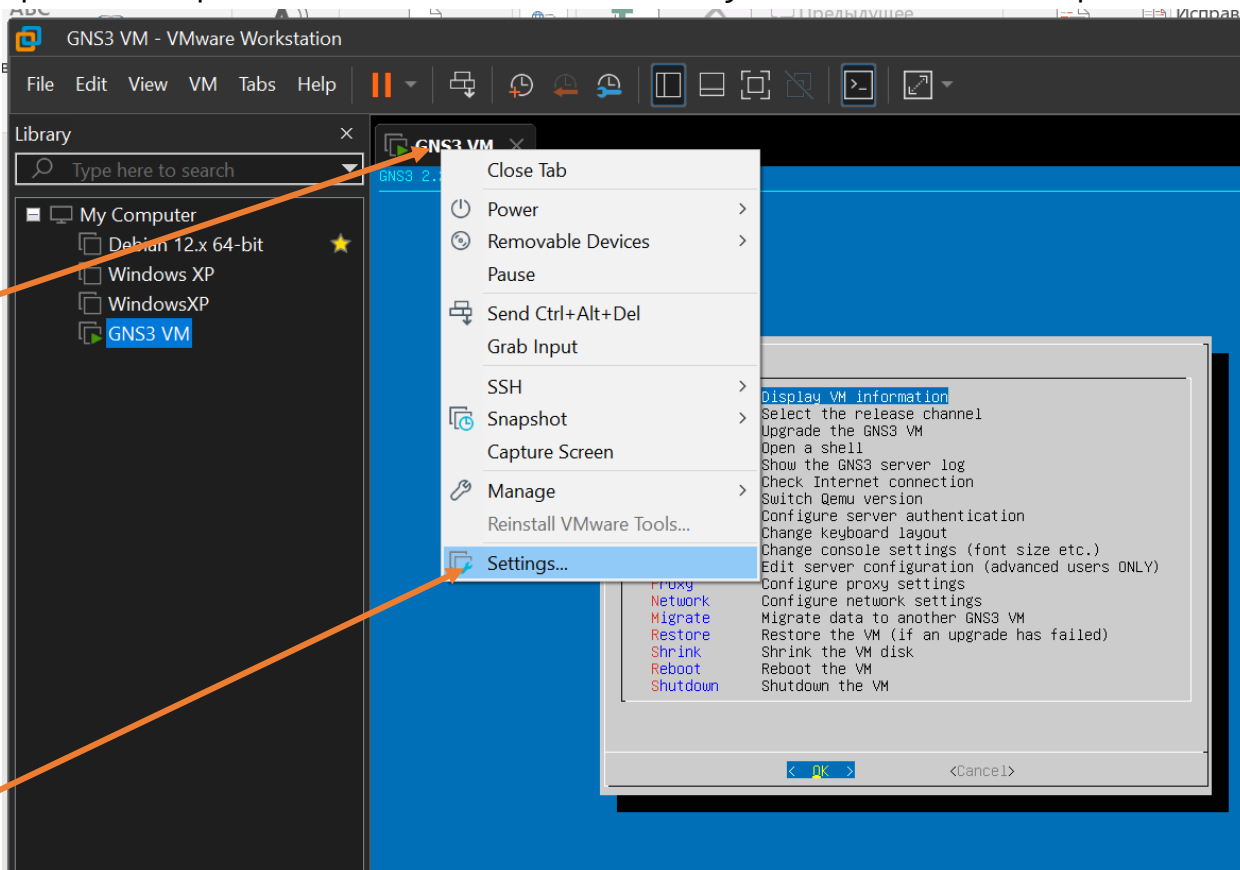


Рисунок 19-8. Настройка сетевых интерфейсов в программе VMware Workstation Pro.

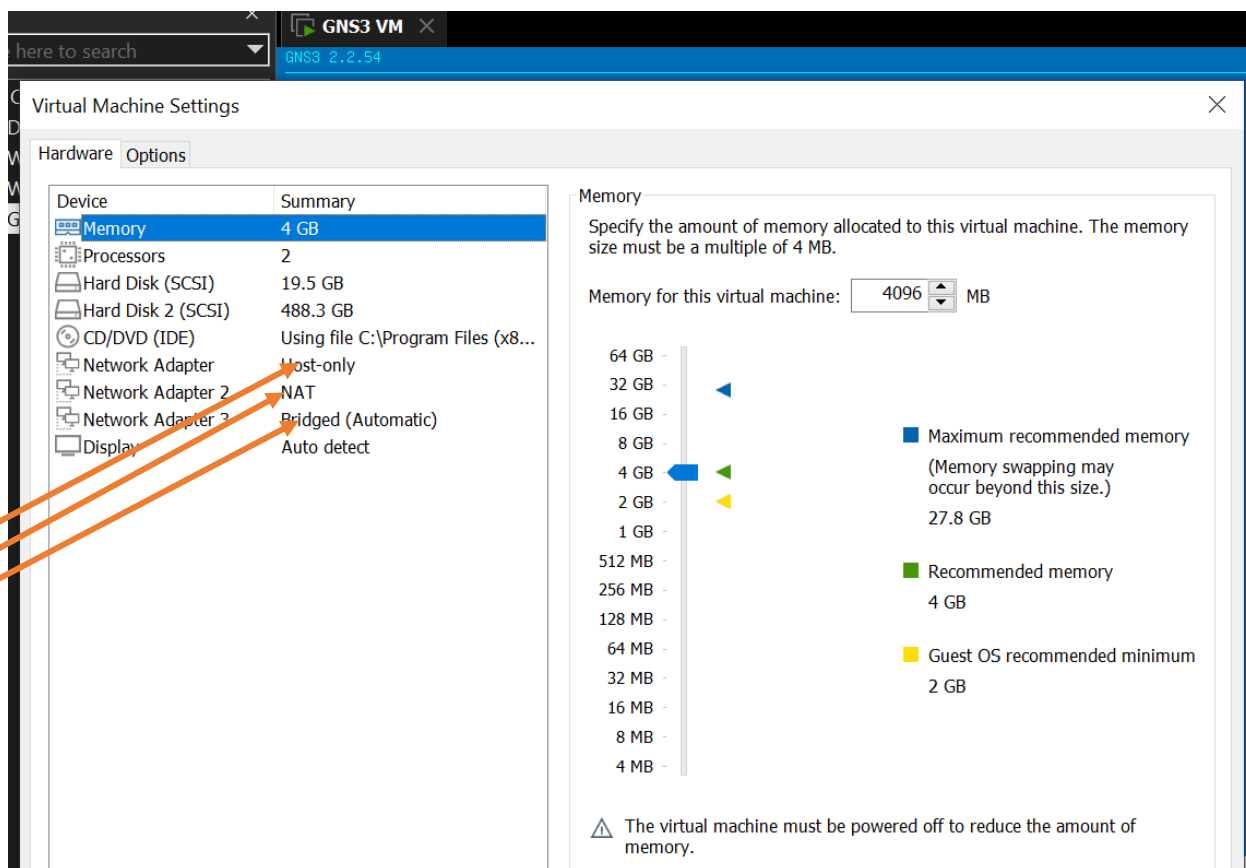


Рисунок 19-9. Настройка сетевых интерфейсов в виртуальной машине GNS3 VM.

Настройка сети QinQ (Q-in-Q) на Debian включает настройку виртуальных интерфейсов для VLAN, созданных с помощью QinQ-инкапсуляции. Это может быть необходимо, например, для маршрутизации трафика между несколькими VLAN или обеспечения присутствия сервера в нескольких VLAN одновременно.

Настройка может осуществляться в ядре Linux или с помощью службы NetworkManager.

В ядре

Необходима поддержка 802.1Q ядром Linux. Если модуль не найден, необходимо переконфигурировать ядро, включив поддержку модуля, а потом пересобрать модули. Модуль включается в разделе «Network options» / «802.1Q VLAN Support».

Некоторые шаги настройки:

- **Создать виртуальные интерфейсы** с именами, содержащими VLAN ID. Например, eth0.2.
- **Назначить каждому интерфейсу свой IP-адрес.**
- **Если маршрут по умолчанию смотрит в один из VLAN**, нужно его задать.
- **Запретить пересылку трафика** между интерфейсами (forwarding) — если пересылку разрешить, весь трафик между VLAN может пересылаться через эту систему.

xgu.ru

Важно: трафик нижележащего интерфейса (например, eth0) будет отсылаться без тега, добавление дополнительных VLAN не скажется на его работе.

Проверка настройки Q-in-Q в Debian 12-есть модуль в ядре:

Чтобы проверить поддержку 802.1Q в ядре Linux, можно использовать команду **modprobe 8021q**. Если модуль уже загружен, появится ошибка: modprobe: ERROR: could not insert '8021q': Module already in kernel

```
sudo modprobe 8021q
```

Но сначала подключим виртуальную машину к интернет и обновим пакеты:

```
root@debian:/home/debian# cat /etc/network
network/ networks
root@debian:/home/debian# cat /etc/network/interfaces
# This file describes the network interfaces available on your
system
# and how to activate them. For more information, see
interfaces(5).
```

```
source /etc/network/interfaces.d/*

# The loopback network interface
auto lo
iface lo inet loopback

# DHCP config for ens4
#auto ens4
iface ens4 inet dhcp

# Static config for ens4
#auto ens4
iface ens4 inet static
# address 192.168.1.100
# netmask 255.255.255.0
# gateway 192.168.1.1
# dns-nameservers 192.168.1.1
root@debian:/home/debian#
debian@debian:~$ sudo modprobe 8021q
[ 424.830118] 8021q: 802.1Q VLAN Support v1.8
[ 424.835368] 8021q: adding VLAN 0 to HW filter on device ens4
debian@debian:~$
```

Нужно снять комментарии с описания интерфейса ens4-конфигурация сетевого подключения должна быть такой:

```
# DHCP config for ens4
auto ens4
iface ens4 inet dhcp
```

Перезагружаем сеть и обновляем пакеты:

```
root@debian:/home/debian# systemctl restart networking
root@debian:/home/debian# ping 8.8.8.8 -c 3
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp seq=1 ttl=106 time=27.4 ms
64 bytes from 8.8.8.8: icmp seq=2 ttl=106 time=29.1 ms
64 bytes from 8.8.8.8: icmp seq=3 ttl=106 time=32.0 ms
```

--- 8.8.8.8 ping statistics ---

3 packets transmitted, 3 received, 0% packet loss, time 2005ms

rtt min/avg/max/mdev = 27.388/29.510/32.031/1.916 ms

root@debian:/home/debian#

root@debian:/home/debian# apt update && apt upgrade -y

Чтобы убедиться, что модуль загружен, нужно выполнить команду `lsmod | grep 8021q`.
Если в результате появится вывод, например:

8021q 33080 0 garp 14384 1 8021q mrp 18542 1 8021q

, то модуль 8021q имеется.

sudo lsmod | grep 8021q

debian@debian:~\$ sudo lsmod | grep 8021q

8021q 40960 0

garp 16384 1 8021q

mrp 20480 1 8021q

debian@debian:~\$

Элемент Cloud1 на схеме при начальной конфигурации следует привязать для исполнения в виртуальной машине GNS3.VM.

Настроить встроенные коммутаторы GNS3 для пропуска пакетов из локальной сети в виртуальную машины с Дебиан в соответствии с рисунками Рис 19-8 и Рис 19-9:

Switch4 configuration

General

Name: Switch4

Console type: none

Settings

Port: 8

VLAN: 1

Type: access

QinQ EtherType: 0x8100

Ports

Port	VLAN	Type	EtherType
0	1	dot1q	
1	100	qinq	0x8100
2	1	access	
3	1	access	
4	1	access	
5	1	access	
6	1	access	
7	1	access	

Add Delete

Рисунок 19-8. Настройка свича с QinQ

Switch6 configuration

General

Name:

Console type:

Settings

Port:

VLAN:

Type:

QinQ EtherType:

Ports

Port	VLAN	Type	EtherType
0	1	dot1q	
1	10	access	
2	20	access	
3	30	access	
4	1	access	
5	1	access	
6	1	access	
7	1	access	

Рисунок 29-9. Настройка свича доступа.

Сделать обновление пакетов на всех машинах:

```
apt install --reinstall ifupdown netplan.io bridge-utils  
python3-netifaces  
apt update && apt upgrade -y
```

Для удобства работы заменим имена на роутерах:

```
debian@R2:~$ cat /etc/hostname  
R1  
debian@R2:~$
```

```
debian@R2:~$ cat /etc/hostname  
R2  
debian@R2:~$
```

Здесь R1 – левый роутер и R2 – правый роутер на схеме.

Создадим интерфейсы в соответствии со схемой на рисунке 19-6.

```
root@R1:/home/debian# cat /etc/network/interfaces  
auto lo  
iface lo inet loopback  
  
auto ens4  
iface ens4 inet dhcp  
  
# trunk ports
```


auto ens5
iface ens5 inet manual

auto ens6
iface ens6 inet manual

Outer VLAN tag (S-tag 100)

auto ens5.100
iface ens5.100 inet manual
 vlan-raw-device ens5
 vlan-protocol 802.1ad
 vlan-id 100

auto ens6.100
iface ens6.100 inet manual
 vlan-raw-device ens6
 vlan-protocol 802.1ad
 vlan-id 100

Inner VLAN10

auto ens5.100.10
iface ens5.100.10 inet manual
 vlan-raw-device ens5.100
 vlan-id 10

auto ens6.100.10
iface ens6.100.10 inet manual
 vlan-raw-device ens6.100
 vlan-id 10

Inner VLAN20

auto ens5.100.20
iface ens5.100.20 inet manual
 vlan-raw-device ens5.100
 vlan-id 20

```
auto ens6.100.20  
iface ens6.100.20 inet manual  
    vlan-raw-device ens6.100  
    vlan-id 20  
  
# Bridge for VLAN10  
auto br10  
iface br10 inet static  
    address 172.16.1.1  
    netmask 255.255.255.0  
    bridge-ports ens5.100.10 ens6.100.10  
    bridge-stp off  
    bridge-fd 0  
    bridge hw 0c:9c:f0:9b:31:31  
  
# Bridge for VLAN20 (no address)  
auto br20  
iface br20 inet static  
    address 192.168.20.1  
    netmask 255.255.255.0  
    bridge-ports ens5.100.20 ens6.100.20  
    bridge-stp off  
    bridge-fd 0  
    bridge hw 0c:9c:f0:9b:32:32  
root@R1:/home/debian#
```

Внимание! Бриджы должны иметь явно прописанные и уникальные MAC адреса. По умолчанию они получают одинаковые и будет ошибка.

На каждом следующем устройстве с Дебиан адреса br10 , br20 изменяем увеличивая на 1.

Точно такой же файл пишем на втором маршрутизаторе с Debian в правой части схемы.

И рестарт сетевой сервис.

Некоторые параметры настройки моста :

bridge_stp выключен # отключить протокол связующего дерева

bridge_waitport 0 # нет задержки до того, как порт станет доступным

bridge_fd 0 # нет задержки

при пересылке bridge_ports нет # если вы не хотите привязываться к каким-либо портам
, используйте регулярное выражение bridge_ports eth* # используйте регулярное
выражение для определения портов.

Обращаем внимание на ошибки в консоли устройств R1 и R2:

4. Особенность Debian: конфликт systemd-networkd

В Debian 12 **одновременно включены** два сетевых механизма:

- ifupdown → использует /etc/network/interfaces
- systemd-networkd → автоматически поднимает интерфейсы из /sys/class/net

Из-за этого Debian дублирует создание VLAN-интерфейсов и выдает ошибки:

```
Error: 8021q: VLAN device already exists.  
ifup: ignoring unknown interface ens5.100.10
```

Решение

Полностью отключить systemd-networkd:

```
systemctl disable --now systemd-networkd  
systemctl disable --now systemd-networkd-wait-online  
systemctl mask systemd-networkd  
systemctl mask systemd-networkd.socket
```

После этого:

```
systemctl restart networking
```

Ошибки исчезают.

5. Проверка работы широковещательного домена

Для проверки работы полученного с помощью протокола QinQ широковещательного домена можно использовать сервер DHCP. Настроим его на виртуальной машине с именем R1:

Устанавливаем пакет с сервером:

```
apt install isc-dhcp-server
```

Проверяем статус:

```
systemctl status isc-dhcp-server
root@R1:/etc/default# systemctl status isc-dhcp-server | more
• isc-dhcp-server.service - LSB: DHCP server
    Loaded: loaded (/etc/init.d/isc-dhcp-server; generated)
    Active: active (running) since Tue 2025-12-02 09:52:48 UTC;
18min ago
    Docs: man:systemd-sysv-generator(8)
    Process: 851 ExecStart=/etc/init.d/isc-dhcp-server start
(code=exited, statu
s=0/SUCCESS)
    Tasks: 1 (limit: 2348)
    Memory: 6.5M
    CPU: 100ms
    CGroup: /system.slice/isc-dhcp-server.service
           └─868 /usr/sbin/dhcpd -4 -q -cf /etc/dhcp/dhcpd.conf
br10 br20

Dec 02 09:52:46 R1 dhcpd[864]: All rights reserved.
Dec 02 09:52:46 R1 dhcpd[864]: For info, please visit
https://www.isc.org/softwa
re/dhcp/
Dec 02 09:52:46 R1 dhcpd[868]: Internet Systems Consortium DHCP
Server 4.4.3-P1
Dec 02 09:52:46 R1 dhcpd[868]: Copyright 2004-2022 Internet
Systems Consortium.
Dec 02 09:52:46 R1 dhcpd[868]: All rights reserved.
Dec 02 09:52:46 R1 dhcpd[868]: For info, please visit
https://www.isc.org/softwa
re/dhcp/
Dec 02 09:52:46 R1 dhcpd[868]: Wrote 5 leases to leases file.
Dec 02 09:52:46 R1 dhcpd[868]: Server starting service.
Dec 02 09:52:48 R1 isc-dhcp-server[851]: Starting ISC DHCPv4
server: dhcpd.
Dec 02 09:52:48 R1 systemd[1]: Started isc-dhcp-server.service -
LSB: DHCP serve
r.
```

Состояние должно быть Active как на листинге выше.

Конфигурационные файлы сервера DHCP находятся в

```
root@R1:/# ls -al /etc/default/isc-dhcp-server
-rw-r--r-- 1 root root 634 Nov 20 13:15 /etc/default/isc-dhcp-server

root@R1:/# ls -al /etc/dhcp/dhcpd.conf
-rw-r--r-- 1 root root 3556 Nov 17 07:20 /etc/dhcp/dhcpd.conf

root@R1:/#
```

В руководствах по настройке сервера рекомендуется сделать копии дефолтных конфигов:

```
cp /etc/default/isc-dhcp-server /etc/default/isc-dhcp-server.bak
cp /etc/dhcp/dhcpd.conf /etc/dhcp/dhcpd.bak.conf
```

Содержание конфигурационных файлов должно быть таким для нашей задачи:

```
root@R1:/# cat /etc/default/isc-dhcp-server
# Defaults for isc-dhcp-server (sourced by /etc/init.d/isc-dhcp-server)

# Path to dhcpd's config file (default: /etc/dhcp/dhcpd.conf).
#DHCPDv4 CONF=/etc/dhcp/dhcpd.conf
#DHCPDv6 CONF=/etc/dhcp/dhcpd6.conf

# Path to dhcpd's PID file (default: /var/run/dhcpd.pid).
#DHCPDv4 PID=/var/run/dhcpd.pid
#DHCPDv6 PID=/var/run/dhcpd6.pid

# Additional options to start dhcpd with.
# Don't use options -cf or -pf here; use DHCPD CONF/DHCPD PID instead
#OPTIONS=""

# On what interfaces should the DHCP server (dhcpd) serve DHCP requests?
# Separate multiple interfaces with spaces, e.g. "eth0 eth1".
INTERFACESv4="br10 br20"
INTERFACESv6=""

root@R1:/#
```

строка, отвечающая за прослушивание нужных интерфейсов с пакетами запросов на адреса выделена рамкой и подчеркиванием.

```
root@R1:/# cat /etc/dhcp/dhcpd.conf | grep -v '#'
option domain-name-servers 8.8.8.8, 77.88.8.8;
default-lease-time 600;
```

```

max-lease-time 7200;
ddns-update-style none;
authoritative;
log-facility local7;
subnet 172.16.1.0 netmask 255.255.255.0 {
    range 172.16.1.10 172.16.1.30;
    option routers 172.16.1.1;
}
subnet 192.168.20.0 netmask 255.255.255.0 {
    range 192.168.20.10 192.168.20.30;
    option routers 192.168.20.1;
}
root@R1:/#

```

Адреса для устройств из виртуальной сети Vlan10 будет выдаваться из блока 172.16.1.10-172.16.1.30, а для Vlan20 из блока 192.168.20.10-192.168.20.30 в соответствии с запросами из интерфейсов с именами br10 , br20 и их адресами.

Список выданных адресов можно посмотреть командой:

```

root@R1:/# dhcp-lease-list
To get manufacturer names please download http://standards-oui.ieee.org/oui.txt to /usr/local/etc/oui.txt
Reading leases from /var/lib/dhcp/dhcpd.leases

```

MAC	IP	hostname	valid until
00:50:79:66:68:00	172.16.1.16	PC21	2025-12-02
10:57:40 -NA-			
00:50:79:66:68:03	192.168.20.13	PC51	2025-12-02
10:59:58 -NA-			
0c:9c:f0:9b:31:31	172.16.1.17	R1	2025-12-02
10:56:02 -NA-			
0c:9c:f0:9b:32:32	192.168.20.15	R1	2025-12-02
10:56:05 -NA-			
172.16.1.16 dev br10 lladdr 00:50:79:66:68:00 STALE			
fe80::52ff:20ff:feae:a702 dev ens4 lladdr 50:ff:20:ae:a7:02			
router REACHABLE			
2001:470:dd07:0:52ff:20ff:feae:a702 dev ens4 lladdr			
50:ff:20:ae:a7:02 router REACHABLE			

Для проверки запросов к серверу со стороны включим захват пакетов на физическом интерфейсе маршрутизатора R1 с помощью утилиты TCPDUMP:

```
root@R1:/home/debian# tcpdump -vvv -i ens5 -e -n vlan
tcpdump: listening on ens5, link-type EN10MB (Ethernet), snapshot
length 262144 bytes
```

А в локальной сети за маршрутизатором R2 запустим на ПК VPC2 запрос на получение адреса от сервера DHCP:

```
PC2> ip dhcp
DORA IP 172.16.1.16/24 GW 172.16.1.1
```

В листинге дампа захвата пакетов видно, что успешно широковещательные пакеты приходят на сервер и он выдает адреса.

```
04:04:17.259832 00:50:79:66:68:00 > ff:ff:ff:ff:ff:ff, ethertype
802.1Q (0x8100), length 414: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype IPv4 (0x0800), (tos 0x10, ttl 16, id 0, offset
0, flags [none], proto UDP (17), length 392)
```

```
0.0.0.0.68 > 255.255.255.255.67: [udp sum ok] BOOTP/DHCP,
Request from 00:50:79:66:68:00, length 364, xid 0x7b7c969, Flags
[none] (0x0000)
```

```
Client-Ethernet-Address 00:50:79:66:68:00
```

```
Vendor-rfc1048 Extensions
```

```
Magic Cookie 0x63825363
```

```
DHCP-Message (53), length 1: Discover
```

```
Hostname (12), length 4: "PC21"
```

```
Client-ID (61), length 7: ether 00:50:79:66:68:00
```

```
END (255), length 0
```

```
PAD (0), length 0, occurs 105
```

```
04:04:17.261292 0c:9c:f0:9b:31:31 > 00:50:79:66:68:00, ethertype
802.1Q (0x8100), length 350: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype IPv4 (0x0800), (tos 0x10, ttl 128, id 0,
offset 0, flags [none], proto UDP (17), length 328)
```

```
172.16.1.1.67 > 172.16.1.16.68: [udp sum ok] BOOTP/DHCP,
Reply, length 300, xid 0x7b7c969, Flags [none] (0x0000)
```

```
Your-IP 172.16.1.16
```

```
Client-Ethernet-Address 00:50:79:66:68:00
```

```
Vendor-rfc1048 Extensions
```

```
Magic Cookie 0x63825363
```

```

DHCP-Message (53), length 1: Offer
Server-ID (54), length 4: 172.16.1.1
Lease-Time (51), length 4: 468
Subnet-Mask (1), length 4: 255.255.255.0
Default-Gateway (3), length 4: 172.16.1.1
Domain-Name-Server (6), length 8: 8.8.8.8,77.88.8.8
END (255), length 0
PAD (0), length 0, occurs 22
04:04:18.260645 00:50:79:66:68:00 > 0c:9c:f0:9b:31:31, ethertype
802.1Q (0x8100), length 414: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype IPv4 (0x0800), (tos 0x10, ttl 16, id 0, offset
0, flags [none], proto UDP (17), length 392)
0.0.0.0.68 > 255.255.255.255.67: [udp sum ok] BOOTP/DHCP,
Request from 00:50:79:66:68:00, length 364, xid 0x7b7c969, Flags
[none] (0x0000)
Client-IP 172.16.1.16
Client-Ethernet-Address 00:50:79:66:68:00
Vendor-rfc1048 Extensions
Magic Cookie 0x63825363
DHCP-Message (53), length 1: Request
Server-ID (54), length 4: 172.16.1.1
Requested-IP (50), length 4: 172.16.1.16
Client-ID (61), length 7: ether 00:50:79:66:68:00
Hostname (12), length 4: "PC21"
Parameter-Request (55), length 4:
Subnet-Mask (1), Default-Gateway (3), Domain-Name-
Server (6), Domain-Name (15)
END (255), length 0
PAD (0), length 0, occurs 87
04:04:18.261428 0c:9c:f0:9b:31:31 > 00:50:79:66:68:00, ethertype
802.1Q (0x8100), length 350: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype IPv4 (0x0800), (tos 0x10, ttl 128, id 0,
offset 0, flags [none], proto UDP (17), length 328)
172.16.1.1.67 > 172.16.1.16.68: [udp sum ok] BOOTP/DHCP,
Reply, length 300, xid 0x7b7c969, Flags [none] (0x0000)
Client-IP 172.16.1.16
Your-IP 172.16.1.16
Client-Ethernet-Address 00:50:79:66:68:00
Vendor-rfc1048 Extensions

```



```

      Magic Cookie 0x63825363
      DHCP-Message (53), length 1: ACK
      Server-ID (54), length 4: 172.16.1.1
      Lease-Time (51), length 4: 467
      Subnet-Mask (1), length 4: 255.255.255.0
      Default-Gateway (3), length 4: 172.16.1.1
      Domain-Name-Server (6), length 8: 8.8.8.8,77.88.8.8
      END (255), length 0
      PAD (0), length 0, occurs 22
04:04:19.270040 00:50:79:66:68:00 > ff:ff:ff:ff:ff:ff, ethertype
802.1Q (0x8100), length 72: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype ARP (0x0806), Ethernet (len 6), IPv4 (len 4),
Request who-has 172.16.1.16 (ff:ff:ff:ff:ff:ff) tell 172.16.1.16,
length 50
04:04:20.282768 00:50:79:66:68:00 > ff:ff:ff:ff:ff:ff, ethertype
802.1Q (0x8100), length 72: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype ARP (0x0806), Ethernet (len 6), IPv4 (len 4),
Request who-has 172.16.1.16 (ff:ff:ff:ff:ff:ff) tell 172.16.1.16,
length 50
04:04:21.295457 00:50:79:66:68:00 > ff:ff:ff:ff:ff:ff, ethertype
802.1Q (0x8100), length 72: vlan 100, p 0, ethertype 802.1Q (0x8100),
vlan 10, p 0, ethertype ARP (0x0806), Ethernet (len 6), IPv4 (len 4),
Request who-has 172.16.1.16 (ff:ff:ff:ff:ff:ff) tell 172.16.1.16,
length

```

Этапы работы протокола DHCP :

Discover-Offer-Request-Ask выделены рамкой и подчеркиванием.

Так же виден запрос ARP и выделены S-Tag и C-Tag в пакететах.

Задача решена.

6. Почему при tcpdump видны оба тега?

Это закономерное поведение Linux.

Когда кадр приходит на физический интерфейс (ens5):

```
[ Ether ] [ S-tag 100 ] [ C-tag 10 ] [ Payload ]
```

Linux сначала полностью принимает кадр, затем:

1. проверяет 802.1ad или 802.1q по outer-tag

2. передает его в соответствующий интерфейс `ens5.100`
3. затем удаляет второй тег и передает в `ens5.100.10`
4. и только потом — в bridge `br10`

И только bridge видит уже **очищенный** кадр.

Но tcpdump работает на **raw-socket уровне ядра**, до удаления тегов.

Поэтому логично:

- В Debian/GNS3 видны S-tag + C-tag
- На Eltex vESR/GN3 ASIC — виден только один тег

7. Ограничения Linux QinQ

Linux поддерживает только **basic QinQ**, то есть просто стекает теги.

Он *не* поддерживает:

- service-delimiting QinQ
- Ether-type rewriting
- S-tag based forwarding decision
- PE-модели «per-service» как у провайдеров (Cisco ME-3600, Eltex MES-5324)

Таким образом Debian работает как простой QinQ switch/bridge.

8. Результаты практики

- QinQ в Debian работает стабильно
- DHCP успешно проходит через двухуровневый тег
- можно построить L2VPN между R1 и R2
- tcpdump показывает все теги
- systemd-networkd обязательно нужно отключить

9. Заключение

Debian 12 — отличный инструмент для лабораторий по QinQ, если понимать его отличия от L2-устройств с ASIC. Он позволяет наблюдать полный состав тегов, что делает его полезным учебным стендом.

Решение, приведённое в этой главе, можно использовать для:

- изучения QinQ

- построения L2VPN
- тестирования DHCP/PPP/L2TP поверх тегов
- отладки GNS3-лабораторий